

Travaux Pratiques 1 Régression linéaire multiple

Contents

1	Objectifs du TP	2
1.1	Objectif général	2
1.2	Objectifs spécifiques	2
2	Régression sur un jeu de données synthétique	2
2.1	Exercice : Pertinence de la régression	2
2.2	Exercice : Détermination des paramètres de la régression	2
2.3	La bonne méthode	3
3	Application : Régression sur un jeu de données réel	4
3.1	Sélection d'attributs	4
3.2	Apprentissage et évaluation	4
4	Un peu de travail à la maison	4

¹Responsable du cours : Gorgoumack SAMBE

1 Objectifs du TP

1.1 Objectif général

A la fin de cette activité l'apprenant devra être capable de sélectionner, d'appliquer et d'évaluer une régression linéaire sur python.

1.2 Objectifs spécifiques

Pour objectifs spécifiques, vous devrez être capable à la fin de cette activité

1. de sélectionner les attributs pertinents pour l'application d'une régression linéaire multiple.
2. d'appliquer une régression linéaire multiple par la méthode des moindres carrés ordinaires (MCO).
3. d'évaluer une régression linéaire multiple.

2 Régression sur un jeu de données synthétique

Nous avons appris à générer un jeu de données synthétique au TP0. Générez un jeu de données synthétique de taille 100, avec 3 descripteurs (features).

2.1 Exercice : Pertinence de la régression

1. Affichez les coefficients de corrélation (par couple ou la matrice) entre les différentes variables (descripteurs et cibles).
2. Affichez la matrice de corrélation sous forme de carte thermique.
3. Notez les coefficients de corrélation entre chaque descripteur et la cible. Montrent-elles une corrélation entre chaque descripteur et la variable cible?
4. Observez le coefficient de corrélation entre descripteurs deux à deux. Les descripteurs sont-ils corrélés?
5. Dessinez les nuages de points entre chaque descripteur et la variable cible.

2.2 Exercice : Détermination des paramètres de la régression

1. Importons la classe du modèle linéaire et instancions la :

```
from sklearn.linear_model import LinearRegression
modele_non_evalue = LinearRegression()
```

2. Apprenons sur notre jeu de données (fit) et affichons ensuite les coefficients (coef_) et l'ordonnée à l'origine (intercept_) obtenus :

```
modele_non_evalue.fit(X,y)
print(modele_non_evalue.coef_)
print(modele_non_evalue.intercept_)
```

3. Affichons les valeurs prédites par notre modèle :

```
y_predict = modele_non_evalue.predict(X)
print(y_predict)
```

4. Évaluons la qualité de notre modèle (R2):

```
from sklearn.metrics import r2_score
r2 = r2_score(y, y_predict)
print('R2=',r2)
```

Avons nous un bon modèle ?²

Il peut être intéressant de calculer et d'afficher l'erreur moyenne quadratique (MSE) ou sa racine (RMSE) sur le jeu de données:

```
from sklearn.metrics import mean_squared_error
mse = mean_squared_error(y, y_predict)
print('MSE=',mse)
rmse = np.sqrt(mse)
print('RMSE=',rmse)
```

2.3 La bonne méthode

Dans la section précédente, nous avons fait une régression en utilisant tout le jeu de données. Il serait plus judicieux de décomposer notre jeu de données en **jeu d'apprentissage** et **jeu de test** : cela nous permettra de construire un modèle avec le jeu d'apprentissage et de le valider avec le jeu de test.

1. Décomposons notre jeu de données en données d'apprentissage (80%) et en données de test (20%)³.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=5)
```

2. Créez une instance de régression linéaire nommée **modele**.
3. Apprenez avec les données d'apprentissage pour avoir un modèle.
4. Calculez et affichez le R2 et le RMSE du modèle sur les données d'apprentissage.
5. Pour la validation du modèle, nous allons calculer le RMSE sur les données de test en suivant ces étapes :
 - (a) Calculer les valeurs prédites pour le jeu de données de test
 - (b) Calculer le MSE en utilisant les données prédites et les données réels du jeu de test et déduire le RMSE du MSE.
 - (c) comparer le RMSE sur le jeu de test avec le RMSE sur le jeu d'apprentissage.

²avec un modèle de régression linéaire, nous avons aussi le r2 en faisant `r2=modele_non_evalue.score(X,y)`

³Le `random_state` sert à fixer le grain pour pouvoir reproduire l'expérience sur les mêmes données

3 Application : Régression sur un jeu de données réel

Nous avons exploré boston durant le TP0, nous allons essayer de faire un modèle de régression linéaire avec boston. Chargez le jeu de données boston et mettez le dans une dataframe.

3.1 Sélection d'attributs

Lorsque nous souhaitons faire une régression, il est recommandé de faire une sélection sur les attributs pour avoir un "bon modèle" avec le minimum de descripteurs. Pour cela, il existe plusieurs méthodes.

1. Nous sélectionnons pour vous les descripteurs suivants pour l'apprentissage : LSTAT, RM, TAX et PTRATIO. Notez leur coefficient de corrélation avec la cible.
2. Créer un dataframe X avec ces quatre descripteurs et un dataframe Y avec l'attribut PRIX.

3.2 Apprentissage et évaluation

1. Créer un jeu d'apprentissage de 70% et un jeu de test de 30%.
2. Apprendre sur le jeu d'apprentissage et afficher le coefficient de détermination R2.
3. Calculer le RMSE sur le jeu d'apprentissage et sur le jeu de test.
4. Quelles conclusions tirez vous de ces indicateurs?

4 Un peu de travail à la maison

1. sklearn permet de faire une validation croisée (cross_val_score).
Documentez vous et essayer de faire une validation croisée sur boston.
2. sklearn permet également de faire une sélection sur les k attributs les plus pertinents avec la classe SelectKBest en utilisant par exemple Fisher (f_regression).
Documentez vous et essayer de faire une selection sur les 4 attributs les plus pertinents de boston.